

GabrielNat1

**DEVELOPMENT AND ARCHITECTURE OF A WEB-BASED SYSTEM FOR
MANAGING PROFESSIONAL OPPORTUNITIES AND EDUCATIONAL
RESOURCES: A CASE STUDY OF THE 1001 OPPORTUNITIES PROJECT**

Computer Science

2025-2026

Sumário

1 INTRODUCTION	3
2 THEORETICAL FOUNDATION	4
2.1 Web Systems	4
2.2 Software Architecture	4
2.3 Client-Server Architecture	5
2.4 Backend Development	5
2.5 REST APIs	5
2.6 Relational Databases	6
2.7 Web Application Security	6
3 METHODOLOGY	6
4 CHARACTERIZATION OF THE 1001 OPORTUNIDADES PROJECT	7
4.1 Project Objective	7
4.2 Identified Problem and Need	7
4.3 Platform Features	7
5 SYSTEM ARCHITECTURE	7
5.1 General Architecture	7
5.2 Communication Between Components	8
5.3 Separation of Responsibilities	8
6 BACKEND DEVELOPMENT	9
6.1 Java Programming Language	9
6.2 Spring Boot Framework	9
6.3 Application Structure	9
6.4 Controllers	10
6.5 Services	10
6.6 Repositories	10
6.7 Data Transfer Objects	10
6.8 Exception Handling	10
7 DATABASE MODELING	11
7.1 PostgreSQL	11
7.2 Table Organization	11
7.3 Primary and Foreign Keys	11
7.4 Relationships	11
7.5 Data Integrity	12
7.6 Indexes and Optimization	12
8 API AND FRONTEND COMMUNICATION	12
8.1 HTTP Requests	12
8.2 Endpoints	12
8.3 Request Processing	12
9 SYSTEM SECURITY	13
9.1 Authentication	13
9.2 Authorization	13
9.3 Data Validation	13

9.4 Data Protection	13
10 PERFORMANCE AND SCALABILITY	14
10.1 Query Optimization	14
10.2 Pagination	14
10.3 Processing Reduction	14
10.4 Expansion Possibilities	14
11 SYSTEM DEPLOYMENT AND AVAILABILITY	14
12 RESULTS AND DISCUSSION	15
13 FINAL CONSIDERATIONS	15
REFERENCES	16

1. INTRODUCTION

The evolution of the Internet and software development technologies has enabled the creation of systems capable of providing different services through web platforms. Currently, applications of this nature are used for communication, education, commerce, entertainment, information management, and the dissemination of professional opportunities.

However, developing a web application is not limited to creating its visual interface. For a system to be capable of storing information, processing requests, authenticating users, applying business rules, and providing data efficiently, it is necessary to implement a

structure responsible for processing information. This structure is commonly referred to as the backend.

The backend constitutes one of the main components of a web application, acting as an intermediary between the interface used by the user and the services responsible for processing and storing information. An appropriate architecture makes it possible to organize these responsibilities and reduce system complexity.

In this context, the **1001 Oportunidades** project was developed as a web platform designed to provide professional opportunities and educational resources. The platform aims to bring different resources together in a single environment, allowing users to access information related to job opportunities, courses, educational content, and other materials.

The development of the project also aimed to apply knowledge related to Computer Science, particularly programming, backend development, software architecture, databases, and application security.

For the development of the backend, the Java programming language was used together with the Spring Boot framework. Information is stored using the PostgreSQL relational database management system.

The choice of these technologies makes it possible to create an organized application divided into different layers, allowing the input of requests, business rules, and data access to be handled separately.

This work presents the architecture used in the development of the project, covering the foundations of web systems, backend implementation, database modeling, API-based communication, security, performance, and possibilities for future expansion.

2. THEORETICAL FOUNDATION

2.1 Web Systems

Web systems are applications made available through computer networks and generally accessed through web browsers. Unlike applications executed exclusively in local environments, web systems can provide their resources to different users through a server infrastructure.

A web application normally consists of different components responsible for specific functions. These include the user interface, the server responsible for processing requests, and the database responsible for storing information.

This separation allows each component to be developed and maintained relatively independently.

In the 1001 Oportunidades project, this separation is used to divide the application into frontend, backend, and database components.

2.2 Software Architecture

Software architecture represents the organization of the main components of a system and the way in which these components interact with one another.

A well-defined architecture makes it possible to identify the responsibilities of each part of the application and establish how these components communicate.

In the development of more complex systems, the absence of an organized architecture can result in code that is difficult to understand, test, maintain, and modify.

For this reason, the backend of 1001 Oportunidades uses an organization based on the separation of responsibilities.

2.3 Client-Server Architecture

Client-server architecture divides a system into two primary components.

The client is responsible for interacting with the user and sending requests to the server. The server receives these requests, performs the necessary processing, and returns a response.

In a web application, the browser or frontend can be considered the client, while the backend acts as the server.

In 1001 Oportunidades, the simplified communication flow can be represented as:

Client → API → Backend → Database

This architecture prevents the client from needing direct knowledge of the internal structure of the database.

2.4 Backend Development

The backend corresponds to the part of the system responsible for processing operations performed by the application.

Its responsibilities include:

- request processing;
- implementation of business rules;
- user authentication;
- operation authorization;
- database communication;
- data validation;
- data processing;
- API availability.

In the analyzed project, these functions are implemented using Java and Spring Boot.

2.5 REST APIs

An API, or Application Programming Interface, defines mechanisms through which different software components can communicate.

In web applications, HTTP-based APIs allow the frontend to request information and perform operations through the backend.

Among the main HTTP methods are:

GET: generally used to retrieve information.

POST: generally used to create new records.

PUT: generally used to update existing information.

DELETE: generally used to remove information.

Through these methods, the frontend can interact with resources provided by the backend.

2.6 Relational Databases

Relational databases organize information into structures known as tables. These tables can establish relationships with one another.

This model makes it possible to represent different entities within an application and establish relationships between them.

PostgreSQL was selected to perform this function in the 1001 Oportunidades project.

2.7 Web Application Security

Security must be considered throughout the development process of an application.

Systems that store user information need to prevent data from being accessed or modified by unauthorized individuals.

Common mechanisms used in web applications include authentication, authorization, data validation, and access control.

3. METHODOLOGY

The project was developed incrementally, allowing different features to be implemented, tested, and improved throughout the development process.

Initially, the platform's needs and the resources that should be made available to users were identified. Subsequently, the technologies and general system structure were defined.

The backend was developed using Java and Spring Boot, while the database was structured using PostgreSQL.

Throughout development, aspects related to code organization, data persistence, security, performance, and future expansion were considered.

The methodology also involved testing and correction during implementation, allowing problems to be identified and changes to be made to the system structure.

4. CHARACTERIZATION OF THE 1001 OPORTUNIDADES PROJECT

4.1 Project Objective

The 1001 Oportunidades project was developed with the objective of creating a platform that brings together professional opportunities and educational resources.

The proposal is to provide different features within a single environment, facilitating users' access to information and educational content.

The platform is available online, allowing its resources to be accessed through the Internet.

4.2 Identified Problem and Need

Searching for professional opportunities and educational resources may require users to access multiple platforms.

The proposal of the project is to bring part of this information together in a single system, creating a platform that combines professional opportunities with resources related to users' education and development.

To meet this objective, it was necessary to create a structure capable of storing different categories of data and processing the operations performed by users.

4.3 Platform Features

The platform provides features related to:

- professional opportunities;
- courses;
- educational videos;
- supporting materials;
- question banks;
- digital library;
- user profiles;
- activity tracking.

The existence of these different features requires a backend structure capable of managing multiple types of information.

5. SYSTEM ARCHITECTURE

5.1 General Architecture

The system architecture can be divided into three main components:

Frontend

Responsible for the visual interface and interaction with the user.

Backend

Responsible for request processing and business rules.

Database

Responsible for persistent information storage.

The architecture can be represented as follows:

User → Frontend → REST API → Spring Boot → PostgreSQL

This separation allows each component to maintain distinct responsibilities.

5.2 Communication Between Components

When the user performs an action on the frontend, an HTTP request is sent to the backend.

The backend receives the request and identifies which operation needs to be performed.

If the operation requires stored information, the backend performs a query against PostgreSQL.

After processing, the result is returned to the frontend.

This flow can be represented as:

Frontend → Controller → Service → Repository → PostgreSQL

and subsequently:

PostgreSQL → Repository → Service → Controller → Frontend

5.3 Separation of Responsibilities

Separation of responsibilities is one of the principles used to organize the backend.

Each layer has a specific function.

The **Controller** is responsible for HTTP communication.

The **Service** is responsible for business rules.

The **Repository** is responsible for data persistence.

This division reduces coupling between components and facilitates system maintenance.

6 BACKEND DEVELOPMENT

6.1 Java Programming Language

Java was used as the primary programming language for backend development.

The language provides object-oriented programming capabilities, allowing the application to be organized through classes and objects.

The use of this paradigm facilitates the representation of entities and components within the system.

In addition, the Java ecosystem provides numerous tools and frameworks designed for enterprise applications and web systems.

6.2 Spring Boot Framework

Spring Boot was used as the foundation for backend development.

The framework simplifies application configuration and allows HTTP services to be provided using Java.

It also integrates with different components commonly used in application development, including persistence systems and dependency injection mechanisms.

Dependency injection allows application components to be provided and managed in an organized manner, reducing the need to manually instantiate all dependencies between classes.

6.3 Application Structure

The backend can be organized using a structure similar to the following:

src/

└─ main/

└─ java/

└─ ...

└─ controller/

└─ service/

|— repository/
|— model/
|— dto/
└— config/

Each directory has a specific responsibility.

This structure facilitates file organization and allows new developers to understand the project more quickly.

6.4 Controllers

Controllers receive HTTP requests and forward operations to the appropriate Services.

A Controller can provide different endpoints associated with a particular resource.

For example, a job API could provide operations for retrieving, creating, updating, and deleting records.

The Controller, however, should not contain all the business rules associated with the operation. Its primary responsibility is to mediate HTTP communication.

6.5 Services

Services represent the business logic layer.

This layer can perform validations, combine information, verify conditions, and determine which operations should be executed.

Using this layer prevents business rules from being duplicated across different Controllers.

6.6 Repositories

Repositories are responsible for communication with the database.

They are used to retrieve and persist information.

Separating this responsibility allows the rest of the application to remain independent from the internal details of data persistence.

6.7 Data Transfer Objects

DTOs, or Data Transfer Objects, can be used to control which information is received by or returned from the API.

This mechanism is important because the internal structure of an entity does not necessarily need to be identical to the structure exposed to the client.

For example, an entity may contain internal information that should not be directly returned to the frontend.

DTOs allow the application to create specific representations for individual operations.

6.8 Exception Handling

Real-world applications must be able to handle unexpected situations.

Potential problems include invalid data, nonexistent records, communication failures, and unauthorized operations.

Exception handling allows these problems to be converted into appropriate responses for the client, preventing internal errors from being exposed in an inappropriate manner.

7. DATABASE MODELING

7.1 PostgreSQL

PostgreSQL is used as the primary database for the project.

Its use allows information to be stored persistently and in a structured manner.

The choice of a relational database is appropriate for the project due to the existence of different entities that have relationships with one another.

7.2 Table Organization

Tables represent different types of information existing within the system.

The data managed by the project includes information related to users, job opportunities, courses, videos, supporting materials, and other resources.

Dividing information into specific tables prevents unrelated data from being concentrated into a single structure.

7.3 Primary and Foreign Keys

Primary keys allow each record in a table to be uniquely identified.

Foreign keys establish relationships between tables.

For example, a record belonging to a progress table may contain references to a user identifier and the identifier of the related educational content.

This structure makes it possible to establish relationships between different parts of the system.

7.4 Relationships

Relationships within the database allow real-world situations to be represented within the system.

A user may be associated with different educational contents, a job opportunity may contain specific information, and a course may contain multiple videos.

These relationships make it possible to retrieve related information through SQL operations.

7.5 Data Integrity

Data integrity is important for preventing inconsistent information from being stored.

Constraints such as primary keys, foreign keys, required values, and uniqueness restrictions help ensure that stored records comply with the rules defined for the database.

7.6 Indexes and Optimization

Indexes can be used to accelerate frequently executed queries.

However, index usage must be carefully planned because each index represents an additional database structure that must also be updated when data is modified.

For this reason, index creation should take into account the application's query patterns.

8. API AND FRONTEND COMMUNICATION

8.1 HTTP Requests

Communication between the frontend and backend occurs through the HTTP protocol.

Each request contains information that allows the backend to identify the requested resource and the data submitted by the client.

The backend processes this information and returns a response.

8.2 Endpoints

Endpoints represent the paths made available by the API to access specific resources.

An API may contain different endpoints for users, job opportunities, courses, and other resources.

Endpoint organization should follow a consistent structure to facilitate API consumption by the frontend.

8.3 Request Processing

A request can follow the process below:

1. The user performs an action.
2. The frontend creates an HTTP request.
3. The Controller receives the request.
4. The Service executes the business rules.
5. The Repository queries or modifies the database.
6. PostgreSQL returns the result.
7. The backend processes the response.
8. The frontend receives the data.

This process occurs within a short period of time and is repeated whenever the user performs an operation that depends on the backend.

9. SYSTEM SECURITY

9.1 Authentication

Authentication allows the system to identify the user accessing the platform.

In applications containing user accounts, authentication is necessary to distinguish authenticated users from users who have not yet logged in.

After authentication, the system can associate operations with the corresponding user account.

9.2 Authorization

Authorization determines which operations a user is allowed to perform.

This distinction is important because being authenticated does not necessarily mean that a user has permission to perform every available operation.

For example, certain administrative operations may be restricted to users with specific permissions.

9.3 Data Validation

Data submitted by the client should not automatically be considered trustworthy.

The backend must validate information before processing it.

These validations may verify data format, size, required fields, and application-specific rules.

9.4 Data Protection

Data protection involves multiple mechanisms.

These include access control, information validation, secure connections, and restrictions on operations performed directly against the database.

Using the backend as an intermediary also reduces the need to expose the persistence infrastructure directly to the client.

10. PERFORMANCE AND SCALABILITY

10.1 Query Optimization

System performance is directly related to the efficiency of database queries.

Poorly structured queries may require unnecessary processing and increase response time.

Therefore, it is important to analyze which information is actually required for each operation and avoid queries that return unnecessary data.

10.2 Pagination

Pagination is a strategy used to limit the number of records returned by a query.

In a platform containing a large number of job opportunities, for example, returning every available job in a single request could result in unnecessary processing and data transfer.

Pagination allows results to be divided into multiple pages.

10.3 Processing Reduction

Performance can also be improved by avoiding unnecessary operations.

The backend should process only the information required to satisfy a request.

Likewise, the database should be queried efficiently.

This concern becomes increasingly important as the number of users and records increases.

10.4 Expansion Possibilities

The architecture used allows new features to be added without necessarily modifying the entire application.

New resources can have their own Controllers, Services, Repositories, and database structures.

This characteristic allows the project to expand gradually while maintaining its architectural organization.

11. SYSTEM DEPLOYMENT AND AVAILABILITY

The 1001 Oportunidades project is available in an online environment, allowing its developed resources to be accessed by users through the Internet.

Making the system available online transforms the project into an application that can be used in a real environment, making it possible to observe issues that may not appear in applications executed exclusively in development environments.

Among these concerns are availability, performance, security, data management, and maintenance.

Therefore, the project goes beyond being merely a programming exercise and represents a practical application of software development concepts.

12. RESULTS AND DISCUSSION

The development of 1001 Oportunidades resulted in the construction of a platform capable of integrating different features related to professional opportunities and educational resources.

The use of an organized architecture made it possible to separate the different responsibilities present within the backend.

The use of Spring Boot made it possible to structure request processing and organize application components. Java provided the foundation for implementing business rules, while PostgreSQL enabled structured information storage.

The separation between Controller, Service, and Repository also contributed to code maintainability.

Another relevant aspect was the need to consider the database not merely as a place for storing information, but as a fundamental part of the system architecture.

Appropriate modeling of tables, relationships, and indexes directly influences data consistency and system performance.

During development, it was also necessary to consider security and access control, demonstrating that a web application must be designed with not only its functionality in mind, but also the manner in which its data will be protected.

Therefore, the project made it possible to apply theoretical knowledge to a practical situation, demonstrating the relationship between programming, software architecture, databases, and web development.

13. FINAL CONSIDERATIONS

The development of the 1001 Oportunidades project made it possible to apply fundamental Computer Science concepts to the development of a functional web application.

The adopted architecture uses Java and Spring Boot for backend development and PostgreSQL for information storage.

Dividing the application into different layers made it possible to separate responsibilities related to HTTP communication, business rules, and data persistence.

This organization contributes to system maintainability and allows new features to be implemented in a more structured manner.

PostgreSQL also plays a fundamental role in the project by maintaining the information used by the platform and allowing relationships to be established between different entities.

In addition to implementing the system's features, the development demonstrated the importance of aspects such as security, performance, data integrity, and scalability.

The project also represents a practical application of knowledge acquired in software development, demonstrating that building a system involves more than writing code. It also requires decisions related to architecture, organization, storage, security, and maintenance.

As future work, the platform may continue to receive new features, performance improvements, additional security mechanisms, database query optimizations, and new resources for users.

Therefore, 1001 Oportunidades remains an evolving project that provides continuous opportunities for applying new knowledge and software development techniques.

14. REFERENCES

MDN WEB DOCS. *HTTP: Hypertext Transfer Protocol*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Accessed: Aug. 9, 2026.

POSTGRES SQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Available at: <https://www.postgresql.org/>. Accessed: Aug. 9, 2026.

SPRING. *Spring Boot*. Available at: <https://spring.io/projects/spring-boot>. Accessed: Aug. 9, 2026.

SPRING. *Spring Framework Documentation*. Available at: <https://spring.io/projects/spring-framework>. Accessed: Aug. 9, 2026.

1001 OPORTUNIDADES. *1001 Oportunidades*. Available at: <https://1001oportunidades.com.br/>. Accessed: Aug. 9, 2026.

